

Laporan Tugas Besar Strategi Algoritma Greedy Tank royale MUsangking



Muhammad Reza Taufiqurrahman
124140205

Rasya Dhiandra Bangsawan
124140163

M alif dasya anwar
124140133

**Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera
2026**

BAB I: Deskripsi Tugas

Robocode Tank Royale adalah permainan pemrograman di mana pemain membuat kode untuk tank virtual (bot) yang akan bertarung secara otomatis di arena hingga tersisa satu bot (Battle Royale). Nama Robocode berasal dari "Robot code" (versi awal), sementara Tank Royale adalah versi berikutnya yang memungkinkan bot bermain melalui internet/jaringan.

Pemain tidak mengendalikan bot secara langsung, tetapi berperan sebagai programmer yang menulis logika/otak bot, mencakup: Cara bergerak, Mendeteksi lawan, Menembak, Bereaksi terhadap kejadian dalam pertempuran

Tugas Besar 1 Strategi Algoritma: Mahasiswa diminta membuat bot dengan menerapkan strategi greedy.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah. Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn

API (Application Programming Interface) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

2. Batas Waktu Giliran

Setiap bot memiliki batas waktu per turn (turn timeout), umumnya 30–50 ms. Jika bot tidak mengirimkan perintah gerakan sebelum waktu habis, maka tidak ada perintah yang dikirim ke server pada turn tersebut. Akibatnya, bot melewatkan turn dan tidak bisa menyesuaikan gerakan atau menembak hingga turn berikutnya.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

- Semakin besar energi tembakan, peluru lebih berat, lebih lambat, namun kerusakan lebih besar dan energi yang diperoleh saat mengenai musuh juga lebih besar (3 kali lipat energi tembakan).
- Peluru berat waktu tempuh ke target lebih lama, risiko meleset lebih tinggi.
- Peluru ringan bergerak lebih cepat, lebih mudah mengenai target, tetapi kerusakan kecil dan energi yang diperoleh sedikit.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain. Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:

- Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.
- Gun digunakan untuk menembakkan peluru dan dapat berputar bersama body atau independen dari body.

- Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama body atau independen dari body.

8. Pergerakan

- Bot dapat bergerak maju dan mundur hingga kecepatan maksimum.
- Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum.
- Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran.
- Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

- Turret (meriam) dan radar dapat berputar independen satu sama lain serta independen dari tubuh bot.
- Jika tidak diputar, turret dan radar akan mengarah sama dengan tubuh bot.
- Kecepatan putar per giliran:
 - Radar: 45° (360° dalam 8 giliran)
 - Turret/meriam: 20°
 - Tubuh tank: 10° (dalam kondisi terbaik)
- Kecepatan putar tubuh tank bergantung pada kecepatan gerak bot: semakin cepat bergerak, semakin lambat kemampuannya berbelok.
- Tidak ada energi yang dikonsumsi saat bergerak atau berbelok.

10. Pemindaian

- Radar dapat mendeteksi bot musuh dalam jangkauan hingga 1200 piksel.
- Musuh di luar 1200 piksel tidak terdeteksi.
- Bot hanya bisa memindai musuh yang berada dalam sudut pemindaian (scan arc).
- Sudut pemindaian adalah sapuan radar dari arah sebelumnya ke arah saat ini dalam satu giliran.
- Jika radar tidak bergerak (arah sama dengan giliran sebelumnya), sudut pemindaian = 0° , maka tidak ada musuh yang terdeteksi.
- Saran: Selalu ubah arah radar agar tetap dapat memindai musuh.

11. Skor

Skor akhir bot adalah akumulasi dari 6 komponen dibawah yang nanti akan nanti bot skor dengan skor terbanyak akan jadi pemenangnya

- Bullet Damage = damage peluru ke musuh.

- Bullet Damage Bonus = +20% damage jika peluru membunuh musuh.
- Survival Score = +50 poin setiap bot lain mati.
- Last Survival Bonus = +10 poin × jumlah musuh (untuk bot terakhir yang bertahan).
- Ram Damage = 2 × damage tabrakan ke musuh.
- Ram Damage Bonus = +30% damage jika tabrakan membunuh musuh.

BAB II: Dasar Teori

Algoritma tamak atau dalam bahasa Inggris ***greedy algorithm*** adalah algoritma apa pun yang mengikuti metode heuristik dalam pemecahan masalah untuk membuat pilihan optimal secara setempat di setiap tahap. Dalam banyak masalah, strategi tamak tidak menghasilkan solusi optimal, tetapi suatu heuristik tamak dapat menghasilkan solusi optimal lokal yang mendekati solusi optimal global dalam jangka waktu yang wajar. (dari wikipedia)

Dalam pengembangan bot untuk Robocode Tank Royale, API yang disediakan membantu dalam mengintegrasikan logika dan strategi ke dalam bot. API ini menyediakan sekumpulan metode dan antarmuka (interface) yang memungkinkan bot berinteraksi langsung dengan lingkungan permainan.

Melalui API tersebut, bot dapat mengakses data sensor untuk:

Mendeteksi posisi musuh

Mengukur jarak

Menentukan kondisi medan tempur secara real time

Selain itu, dokumentasi resmi API robocode dev memberikan panduan lengkap mengenai fungsi-fungsi yang ada, cara pemanggilannya, serta contoh kode untuk mengoperasikan bot.

BAB III: Aplikasi Strategi Greedy

3.1 Fungsi Objektif Umum

Skor pada Robocode Tank royale Secara Umum Dihitung Dengan Rumus Berikut

$$f = \text{BulletDmg} + \text{BulletDmgBonus} + \text{RamDmg} + \text{RamDmgBonus} + \text{SurvScore} + \text{SurvBonus}$$

Terlihat Bahwa ada enam komponen yang bisa di maksimalkan atau dengan kata lain f adalah fungsi objektif untuk mendapatkan skor maksimal melalui keenam komponen tersebut

3.2 Eksplorasi Solusi Greedy

3.2.1 Dragon: Maximizing score by Killcount

Bot Dragon adalah bot yang berfokus pada **memaksimalkan skor melalui eliminasi musuh (kill count)**. Bot menggunakan algoritma Greedy dengan selalu memilih aksi yang memberikan peluang terbesar untuk menyerang target yang sedang terdeteksi. Dragon bergerak mengelilingi arena sambil melakukan scanning radar secara terus-menerus. Ketika musuh ditemukan, radar dan meriam langsung diarahkan ke target, kemudian bot memilih kekuatan tembakan berdasarkan jarak musuh untuk meningkatkan peluang memperoleh kill.

a. Mapping Elemen

1. **Himpunan Kandidat** : Semua aksi yang dapat dilakukan bot (bergerak, berputar, scan radar, mengunci target, dan menembak dengan berbagai firepower).
2. **Himpunan Solusi** : Aksi yang dipilih pada suatu turn (arah gerak, target yang dikunci, arah meriam, dan kekuatan tembakan).
3. **Fungsi Solusi** : Memeriksa apakah aksi yang dipilih dapat menghasilkan peluang serangan terhadap musuh yang terdeteksi.
4. **Fungsi Seleksi** : Memilih aksi yang memberikan peluang kill terbesar saat ini, yaitu mempertahankan target yang sudah ditemukan dan menembaknya dengan firepower yang sesuai jarak.
5. **Fungsi Kelayakan** : Memastikan meriam dapat menembak, target masih berada dalam jangkauan radar, dan pergerakan tidak menyebabkan bot menabrak dinding.
6. **Fungsi Objektif** : Memaksimalkan jumlah kill yang diperoleh sehingga meningkatkan total skor pertandingan.

b. Analisis Efisiensi Solusi

Algoritma Dragon terdiri dari proses movement dan shooting. Setiap turn bot hanya melakukan pengecekan posisi arena, perhitungan jarak target, penguncian radar, dan pemilihan firepower berdasarkan jarak. Seluruh operasi menggunakan perhitungan sederhana sehingga kompleksitas waktunya $O(1)$ dan kebutuhan memorinya $O(1)$.

c. Analisis Efektivitas Solusi

Strategi Dragon efektif ketika musuh berada dalam jangkauan radar karena bot terus mempertahankan target yang telah ditemukan dan melakukan serangan secara konsisten. Pemilihan firepower berdasarkan jarak membantu meningkatkan peluang menghasilkan damage dan memperoleh kill. Namun strategi ini kurang optimal terhadap musuh yang bergerak sangat acak karena Dragon tidak melakukan prediksi posisi target sehingga akurasi tembakan dapat menurun.

3.2.2 Bot Ireng : Maximizing Bullet Damage dan Survival Score

Bot Ireng adalah bot yang berfokus pada **memaksimalkan Bullet Damage** dengan meningkatkan peluang peluru mengenai target. Bot menggunakan algoritma Greedy dengan selalu memilih aksi yang memberikan peluang hit terbesar terhadap musuh yang sedang terdeteksi. Ireng mempertahankan radar lock pada target, memprediksi posisi musuh berdasarkan arah dan kecepatan geraknya, kemudian menembakkan peluru ke posisi prediksi tersebut. Selain itu, bot bergerak menyamping (*strafing*) dan mengubah arah secara berkala untuk mengurangi kemungkinan terkena serangan lawan.

a. Mapping Elemen

1. **Himpunan Kandidat** : Semua aksi yang dapat dilakukan bot (bergerak maju/mundur, mengubah arah, mengunci radar, mengarahkan meriam, dan menembak dengan berbagai firepower).
2. **Himpunan Solusi** : Aksi yang dipilih pada suatu turn (target yang diprioritaskan, posisi prediksi musuh, arah gerak, arah meriam, dan kekuatan tembakan).
3. **Fungsi Solusi** : Memeriksa apakah radar berhasil mendeteksi musuh, posisi prediksi telah dihitung, meriam telah diarahkan ke target, dan aksi penembakan dapat dilakukan.
4. **Fungsi Seleksi** : Memilih aksi yang memberikan peluang hit terbesar dengan memprediksi posisi musuh dan menentukan firepower berdasarkan jarak target.
5. **Fungsi Kelayakan** : Memastikan posisi prediksi masih berada di dalam arena, meriam dapat menembak ($GunHeat = 0$), target masih terdeteksi radar, dan pergerakan tidak keluar arena.

6. **Fungsi Objektif** : Memaksimalkan Bullet Damage yang dihasilkan dari setiap tembakan yang mengenai musuh.

b. Analisis Efisiensi Solusi

Algoritma Ireng terdiri dari proses movement, target prediction, dan shooting. Pada setiap turn, bot menghitung posisi prediksi musuh menggunakan data hasil scan sebelumnya, menentukan firepower berdasarkan jarak, serta mengatur arah radar dan meriam. Seluruh proses hanya melibatkan operasi matematika sederhana sehingga kompleksitas waktunya $O(1)$ dan kebutuhan memorinya $O(1)$.

c. Analisis Efektivitas Solusi

Strategi Ireng efektif terhadap musuh yang memiliki pola gerak relatif konsisten karena prediksi posisi target dapat meningkatkan akurasi tembakan dan menghasilkan Bullet Damage yang lebih besar. Pergerakan strafing dan perubahan arah secara berkala juga membantu bot bertahan lebih lama di arena. Namun, efektivitas strategi ini dapat menurun ketika menghadapi musuh yang bergerak sangat acak atau sering berubah arah secara tiba-tiba karena prediksi posisi menjadi kurang akurat dan peluang tembakan meleset meningkat. Bot ireng juga kesulitan dalam menghadapi bot yang menyerangnya dari jarak yang sangat dekat atau bot yang agresif

3.2.3 Betabot: Maximizing Bullet Hit and Minimizing Damage Taken.

Betabot adalah bot tempur yang mengutamakan keselamatan diri dengan selalu menjaga jarak aman dari musuh sambil terus mengunci posisi musuh menggunakan radar secara konsisten. Dengan posisi jarak jauh yang terjaga, Betabot fokus memberikan kerusakan melalui tembakan peluru yang akurat dan berkelanjutan sepanjang pertandingan.

a. Mapping elemen

- 1) Himpunan Kandidat:** Semua aksi yang dapat dilakukan Betabot pada setiap tick, yaitu bergerak maju atau mundur, memutar bodi, mengunci radar ke arah musuh, mengarahkan meriam, dan menembak dengan kekuatan peluru tertentu.
- 2) Himpunan Solusi:** : Aksi yang dipilih pada suatu tick, yaitu arah gerak orbit mengelilingi musuh, posisi radar yang mengunci target, arah meriam yang mengarah ke musuh, dan kekuatan tembakan yang disesuaikan dengan kondisi saat itu.
- 3) Fungsi Solusi:** Memeriksa apakah aksi yang dipilih berhasil mempertahankan jarak aman dari musuh sekaligus menghasilkan peluang tembakan akurat yang dapat memberikan bullet damage secara konsisten.
- 4) Fungsi Seleksi:** Memilih aksi yang menjaga posisi orbit terbaik terhadap musuh, yaitu mempertahankan jarak aman yang sudah terbentuk dan menembak dengan kekuatan peluru yang sesuai hanya ketika meriam sudah lurus menghadap musuh.

5) Fungsi Kelayakan: Memastikan meriam sudah dingin dan siap menembak, radar masih mengunci posisi musuh dalam jangkauan pemantauan, dan pergerakan orbit tidak menyebabkan Betabot menabrak dinding arena.

6) Fungsi Objektif: Memaksimalkan total bullet damage yang diberikan kepada musuh sehingga meningkatkan peluang menghancurkan musuh dan memenangkan pertandingan.

b. Analisis Efisiensi Solusi

Algoritma Betabot terdiri dari proses radar locking, orbit movement, dan shooting. Pada setiap tick, bot menghitung jarak ke musuh, menentukan sudut orbit yang sesuai berdasarkan jarak tersebut, serta mengatur arah radar dan meriam agar selalu mengunci posisi musuh. Seluruh proses hanya melibatkan operasi matematika sederhana sehingga kompleksitas waktunya $O(1)$ dan kebutuhan memorinya $O(1)$.

c. Analisis Efektivitas Solusi

Strategi Betabot efektif terhadap musuh yang agresif dan cenderung mendekat. karena posisi bot yang menjaga jarak aman membuat Betabot sulit dijangkau sekaligus tetap dapat memberikan bullet damage secara konsisten. Radar locking yang berkelanjutan juga memastikan musuh selalu terpantau sehingga tidak ada kesempatan tembakan yang terlewat. Namun, efektivitas strategi ini dapat menurun ketika menghadapi musuh yang juga menjaga jarak jauh atau memiliki sistem prediksi tembakan yang canggih karena peluru dengan kekuatan tetap pada jarak jauh lebih mudah dihindari oleh musuh. Seperti dalam percobaan ini algoritma betabot menjadi kurang efektif karena musuh cenderung memiliki algoritma menghindar peluru yang baik

3.2.4 Banzai: Maximizing Ram damage dan maximizing Bullet hit

Bot ini berkerja dengan menerapkan strategi seperti tentara jepang kamikaze yang maju kedepan musuh ke sedekat mungkin tanpa memikirkan resiko dan mengutamakan penyerangan musuh. Bot ini akan mengunci satu target dan akan menyerang dan melakukan segala cara untuk mendekata ke arah target dan menyerang nya sampai jarak seminimal mungkin

a. Mapping Elemen

- 1) **Himpunan Kandidat** : Semua aksi yang Tersedia
- 2) **Himpunan Solusi**: aksi yang dipilih
- 3) **Fungsi Solusi** : Memeriksa apakah aksi yang dipilih akan menghasilkan ram damage dan bullet hit terbesar
- 4) **Fungsi Seleksi**:Memilih aksi yang memperkecil jarak bot dengan musuh yang diincar serta mengarahkan tembakan ke prediksi posisi musuh

- 5) **Fungsi Kelayakan:** Memeriksa apakah aksi yang dipilih membuat bot bergerak mendekati musuh yang dikunci dengan jarak paling minimal
- 6) **Fungsi Objektif:** Memaksimalkan ram damage dan bullet hit dengan meminimalkan jarak ke target

B. Analisis Efisien solusi

Algoritma Greedy Movement pada bot Banzai memiliki kompleksitas **$O(1)$** karena hanya menghitung bearing dan jarak ke satu target yang sudah dikunci tanpa evaluasi kandidat apapun. Sedangkan Predictive Shooting memiliki kompleksitas **$O(n)$** dimana n adalah jarak ke musuh dibagi kecepatan peluru, karena mensimulasikan posisi musuh per tick hingga peluru diperkirakan mencapai target. Secara memori, bot ini sangat efisien dengan kompleksitas **$O(1)$** karena hanya menyimpan beberapa variabel primitif tanpa struktur data dinamis, sehingga keseluruhan bot berjalan ringan namun tetap agresif dalam setiap engagement.

C. Analisis Efektifitas Solusi

Bot ini efektif apa bila Musuh Yang dilawan memiliki sedang fokus dengan musuh lain tau tidak memiliki algoritma pergerakan kabur yang baik atau pola algoritma tembakan yang *Swiping* . Namun bot ini pada akhirnya sering terjebak di kondisi mati di awal game meskipun dia mampu memberikan *Ram dmg* dan *Bullet Dmg* yang besar ia cenderung mendapat kan posisi terkeroyok misal ketika ia mengunci musuh bot yang ada di tengah atau ketika bot lain yang menggunakan jarak sebagai senjata menyerang nya ketika dia masih mengunci satu musuh

3.3 Strategi Greedy yang Dipilih

Berdasarkan solusi solusi yang kami sudah dijabarkan pada sub bab 3.2, setiap solusi memiliki kelebihan dan kekurangan karena tiap game juga masing masing dipengaruhi dnegan tempat spawn dan kondisi lawan tertentu. Oleh sebab itu kami memutuskan untuk memilih bot yang memiliki algoritma paling konsisten dalam pendapatan skor yaitu bot **Dragon**. Dikarenakan diii pengujian hanya bot dragon lah yang konsisten terhadap skornya

BAB IV: Implementasi dan pengujian

A. Implementasi 3 solusi alternatif

Bot Irgeng

```
## ALGORITMA Irgeng

Inisialisasi batas dinding (WallMargin = 120)
Inisialisasi arah gerakan (moveDirection = 1)
Inisialisasi penghitung waktu menghindari (dodgeCounter = 0)
Inisialisasi memori data musuh (lastEnemyX = 0, lastEnemyY = 0,
lastEnemyHeading = 0, lastEnemySpeed = 0)

Inisialisasi warna bot (Hitam, Kuning, Biru Langit, Merah Tua)
Pisahkan rotasi body, gun, dan radar agar bergerak independen

Gerakkan bot maju dengan jarak sangat jauh (10000)

WHILE bot masih berjalan DO

    Putar radar ke kanan 360 derajat (mencari musuh)

    // PROSEDUR CEK DINDING
    Periksa apakah bot dekat dengan batas dinding arena:
        kiri    (X kurang dari WallMargin)
        kanan   (X lebih dari Lebar Arena - WallMargin)
        bawah   (Y kurang dari WallMargin)
        atas    (Y lebih dari Tinggi Arena - WallMargin)

    IF dekat salah satu dinding THEN
        Hitung koordinat titik tengah arena
        Hitung sudut arah menuju pusat arena berdasarkan posisi bot saat
        ini
```

```

        Hitung selisih sudut putaran (Normalisasi sudut antara -180 sampai
180 derajat)
        Belok kanan sebesar sudut penyesuaian tersebut
        Gerakkan bot maju sejauh 300
    ENDIF

    Tambah dodgeCounter sebesar 1

    // PERUBAHAN ARAH GERAK PER DETIK (ZIG-ZAG)
    IF dodgeCounter lebih dari 35 THEN
        Reset dodgeCounter menjadi 0
        Balikkan arah gerakan (moveDirection dikali -1)
        Belok kanan sebesar (60 dikali moveDirection)
        Gerakkan bot maju/mundur sejauh (250 dikali moveDirection)
    ENDIF

    // JAGA MOMENTUM BERGERAK
    IF sisa jarak gerakan saat ini sama dengan 0 THEN
        Gerakkan bot maju kembali dengan jarak sangat jauh dikali
moveDirection
    ENDIF

    Eksekusi seluruh perintah (Go)

ENDWHILE

---

KETIKA Mendeteksi Bot Musuh (OnScannedBot)

Hitung jarak ke musuh (`distance`)
Inisialisasi arah musuh (`enemyHeading` = 0)

// KALKULASI VEKTOR DAN KELAJUAN MUSUH
IF lastEnemyX tidak sama dengan 0 ATAU lastEnemyY tidak sama dengan 0 THEN
    Hitung enemyHeading berdasarkan perubahan posisi koordinat X dan Y
terakhir musuh
ENDIF

Hitung kecepatan musuh (`enemySpeed`) menggunakan rumus jarak Pythagoras
dari posisi terakhir

Simpan posisi, arah, dan kecepatan musuh saat ini ke dalam variabel memori

```

```

(lastEnemy)

// PENENTUAN DAYA TEMBAK DAN PREDIKSI TARGET (LINEAR AIMING)
IF distance kurang dari 150 THEN
    Set kekuatan peluru (`firePower` = 3)
ELSE IF distance kurang dari 350 THEN
    Set kekuatan peluru (`firePower` = 2)
ELSE
    Set kekuatan peluru (`firePower` = 1)
ENDIF

Hitung kecepatan peluru (`bulletSpeed` = 20 - (3 dikali firePower))
Hitung estimasi waktu tempuh peluru (`time` = distance dibagi bulletSpeed)

Hitung prediksi koordinat X musuh (`predictedX`) di masa depan menggunakan
fungsi Trigonometri Sinus
Hitung prediksi koordinat Y musuh (`predictedY`) di masa depan menggunakan
fungsi Trigonometri Kosinus

Batasi predictedX dan predictedY agar tidak keluar dari dinding arena
(minimal 18, maksimal ukuran arena - 18)

// EKSEKUSI ROTASI SENJATA DAN RADAR
Hitung selisih sudut meriam menuju titik koordinat prediksi (`gunBearing`)
Putar meriam ke kiri sebesar gunBearing

Hitung selisih sudut radar menuju koordinat asli musuh (`radarBearing`)
Putar radar ke kiri sebesar (radarBearing dikali 2) agar musuh terkunci
rapat

// GERAKAN MENGORBIT (STRAFING)
Hitung sudut tegak lurus mengitari musuh (`strafe` = sudut bodi ke musuh +
90 derajat)
Normalisasi sudut strafe agar berada di rentang -180 sampai 180 derajat
Belok kanan sebesar sudut hasil normalisasi strafe tersebut
Gerakkan bot maju/mundur sejauh (200 dikali moveDirection)

// KEPUTUSAN MENEMBAK
IF panas meriam sama dengan 0 DAN nilai mutlak gunBearing kurang dari 6
derajat THEN
    Tembak dengan kekuatan peluru yang sudah ditentukan (firePower)
ENDIF

```

KETIKA Menabrak Bot Musuh (OnHitBot)

Balikkan arah gerakan (moveDirection dikali -1)

Gerakkan bot mundur sejauh 150

IF panas meriam sama dengan 0 THEN

 Tembak instan dengan kekuatan penuh (3)

ENDIF

KETIKA Menabrak Dinding Arena (OnHitWall)

Balikkan arah gerakan (moveDirection dikali -1)

Gerakkan bot mundur sejauh 200

Belok kanan 120 derajat untuk membuang arah dari dinding

BetaBot

```
## ALGORITMA Betabot

Inisialisasi variabel target (hasTarget = false)
Inisialisasi penghitung waktu hilang (scanTimeout = 0)
Inisialisasi arah gerakan (moveDirection = 1)

Inisialisasi warna bot (Cyan, Dark Blue, White, Light Blue)
Pisahkan rotasi body, gun, dan radar agar bergerak independen

WHILE bot masih berjalan DO
    ...
    IF NOT hasTarget THEN
        Putar radar ke kiri 180 derajat (sapuan mencari musuh)
        Gerakkan bot maju perlahan sejauh 50
    ELSE
        Tambah scanTimeout sebesar 1

        IF scanTimeout lebih dari 4 THEN
            Set status hasTarget menjadi false (musuh hilang)
        ENDIF
    ENDIF

// STRATEGI ANTI-WALL
Periksa posisi koordinat bot (X dan Y) terhadap dinding arena

IF dekat dengan dinding batas arena THEN
    Balikkan arah gerakan (moveDirection dikali -1)
    Belok kanan 45 derajat (buang sudut dari dinding)
    Bergerak maju/mundur sejauh 100 berdasarkan moveDirection
ENDIF
```



```
Eksekusi seluruh perintah (Go)

ENDWHILE

## KETIKA Mendeteksi Bot Musuh (OnScannedBot)

Set status hasTarget menjadi true
Reset scanTimeout menjadi 0

Hitung jarak ke musuh
Hitung selisih sudut radar saat ini dengan posisi musuh (radarBearing)
Hitung sudut bodi robot menuju musuh (bodyBearing)
Hitung sudut meriam menuju musuh (gunBearing)

// 1. LOCK RADAR
Putar radar ke kiri sebesar (radarBearing dikali 1.5) untuk mengunci musuh

// 2. GERAKAN MENGHINDAR (Orbiting)
Inisialisasi sudut pendekatan (approachAngle = 90)

IF jarak ke musuh lebih dari 300 THEN
Set approachAngle menjadi 75 (mendekat perlahan)
ENDIF

IF jarak ke musuh kurang dari 150 THEN
Set approachAngle menjadi 105 (menjauh perlahan)
ENDIF

Belok kanan sebesar (bodyBearing + (approachAngle dikali moveDirection))
Gerakkan bot maju/mundur sejauh 120 berdasarkan moveDirection

// Pengacak Gerakan (Anti-Predictive Aiming)
IF nilai acak kurang dari 0.02 (peluang 2%) THEN
Balikkan arah gerakan (moveDirection dikali -1)
ENDIF

// 3. ARAHKAN GUN & TEMBAK
Putar meriam ke kiri sebesar gunBearing

IF panas meriam sama dengan 0 DAN nilai mutlak gunBearing kurang dari 15
derajat THEN
```

```
Tembak dengan kekuatan peluru 2  
ENDIF
```

```
## KETIKA Terkena Peluru Musuh (OnHitByBullet)
```

```
Balikkan arah gerakan secara instan (moveDirection dikali -1)  
Gerakkan bot maju/mundur sejauh 100 berdasarkan moveDirection
```

Bot Banzai

ALGORITMA Banzai

```
Inisialisasi posisi musuh (enemy = koordinat tengah arena)
Inisialisasi waktu sejak tembakan terakhir (turnsSinceShot = 0)
Inisialisasi jarak terdekat (nearestDistance = nilai maksimal pecahan)
Inisialisasi ID musuh terkunci (lockedBotId = -1)
Inisialisasi mode tabrak (ramMode = false)
Inisialisasi waktu mode tabrak (ramTimer = 0)

Inisialisasi warna bot (Cokelat, Cokelat, Jingga, Kuning)
Pisahkan rotasi body, gun, dan radar agar bergerak independen

Putar radar ke kiri mengarah ke koordinat tengah arena
Putar meriam ke kiri mengarah ke koordinat tengah arena

WHILE bot masih berjalan DO

    Tambah turnsSinceShot sebesar 1

    IF ramMode DAN ramTimer setelah dikurangi 1 kurang dari atau sama
    dengan 0 THEN
        Set status ramMode menjadi false
    ENDIF

    Gerakkan bot maju dengan jarak sangat jauh (999)

    IF turnsSinceShot lebih dari 10 THEN
        Putar radar ke kiri 360 derajat (mencari ulang musuh yang hilang)
    ELSE
        Belok kiri mengarah ke posisi terakhir musuh (terkunci)
    ENDIF

    Lakukan pemindaian ulang radar (Rescan)
    Eksekusi seluruh perintah (Go)
    Reset nilai nearestDistance ke nilai maksimal pecahan

ENDWHILE
```

KETIKA Mendeteksi Bot Musuh (OnScannedBot)

Hitung jarak ke musuh (dist)

// KONDISI 1: JIKA DALAM MODE TABRAK (RAM MODE)

IF ramMode THEN

Set lockedBotId menjadi ID musuh yang terdeteksi

Reset turnsSinceShot menjadi 0

Belok kiri mengarah langsung ke koordinat musuh

Gerakkan bot maju sejauh (dist + 20) untuk menabrak keras

Putar meriam ke kiri mengarah ke koordinat musuh

Tembak dengan kekuatan penuh (3)

Simpan koordinat musuh ke dalam variabel enemy

KELUAR DARI FUNGSI (Selesai)

ENDIF

// KONDISI 2: SELEKSI TARGET

IF lockedBotId tidak sama dengan -1 DAN ID musuh tidak sama dengan lockedBotId THEN

KELUAR DARI FUNGSI (Abaikan musuh lain jika sudah mengunci satu target)

ENDIF

IF lockedBotId sama dengan -1 THEN

IF dist lebih dari atau sama dengan nearestDistance THEN

KELUAR DARI FUNGSI (Abaikan jika ada musuh lain yang lebih dekat)

ENDIF

Set nearestDistance menjadi dist

Set lockedBotId menjadi ID musuh tersebut

ENDIF

// KONDISI 3: PENGUNCIAN DAN PREDIKSI TEMBAKAN (LINEAR/CIRCULAR AIM)

Reset turnsSinceShot menjadi 0

Putar radar ke kiri tepat ke posisi musuh saat ini

Inisialisasi koordinat prediksi (preds = koordinat musuh saat ini)

Inisialisasi selisih waktu (deltaTime = 0)

IF dist kurang dari 80 THEN

Set kekuatan peluru (power = 4)

ELSE IF dist kurang dari 350 THEN

Set kekuatan peluru (power = 1.5)

ELSE

Set kekuatan peluru (power = 0.8)

ENDIF

Hitung kecepatan peluru berdasarkan nilai power

WHILE (deltaTime ditambah 1) dikali kecepatan peluru kurang dari jarak ke koordinat prediksi DO

Simulasi pergerakan musuh ke depan berdasarkan arah dan kecepatannya
Update nilai prediksi preds[0] (X) dan preds[1] (Y)

IF koordinat prediksi menabrak atau keluar dinding arena THEN

Batasi koordinat prediksi agar tetap berada di dalam batas dinding arena (18)

KELUAR DARI LOOP (Break)

ENDIF

ENDWHILE

Putar meriam ke kiri mengarah ke koordinat prediksi hasil kalkulasi

Tembak dengan kekuatan peluru yang sudah ditentukan (power)

Salin koordinat prediksi ke dalam variabel enemy

KETIKA Ada Bot yang Mati (OnBotDeath)

IF ID bot yang mati sama dengan lockedBotId THEN

Reset lockedBotId menjadi -1

Reset nearestDistance ke nilai maksimal pecahan

ENDIF

KETIKA Terkena Peluru Musuh (OnHitByBullet)

Hitung sudut asal peluru terhadap arah radar saat ini (radarTurn)

WHILE radarTurn lebih dari 180 DO

Kurangi radarTurn dengan 360

ENDWHILE

WHILE radarTurn kurang dari -180 DO

Tambahkan radarTurn dengan 360

ENDWHILE

Putar radar ke kiri sebesar radarTurn (mengarahkan radar langsung ke posisi penembak)

Set status ramMode menjadi true

```
Set ramTimer menjadi 60 (aktifkan mode mengamuk/menabrak selama 60 tick)
```

B.Penjelasan dari Solusi Greedy Yang dipilih

Pseudocode Utama (Run)

```
ALGORITMA Dragon

Inisialisasi warna bot
Pisahkan rotasi body, gun, dan radar

Gerakkan bot maju dengan jarak sangat jauh

WHILE bot masih hidup DO

    Putar radar ke kanan 20 derajat

    Periksa apakah bot dekat dinding:
        kiri
        kanan
        atas
        bawah

    IF dekat dinding dan arah menuju dinding THEN
        Belok kanan 90 derajat
    ENDIF

    IF gerakan telah selesai THEN
        Bergerak maju kembali
    ENDIF

    Eksekusi seluruh perintah

ENDWHILE
```

Penjelasan

Fungsi utama bot yang berjalan selama pertandingan berlangsung. Dragon melakukan patroli mengelilingi arena sambil terus memutar radar untuk mencari musuh. Ketika mendekati dinding, bot akan berbelok agar tidak mengalami wall damage.

Pseudocode Saat Musuh Terdeteksi (OnScannedBot)

```
PROSEDUR OnScannedBot
```

```
Hitung jarak musuh
```

```
Hitung sudut meriam menuju musuh
```

```
Putar meriam ke arah musuh
```

```
Hitung sudut radar menuju musuh
```

```
Putar radar ke arah musuh
```

```
IF jarak < 150 THEN
```

```
    firePower ← 3
```

```
ELSE IF jarak < 400 THEN
```

```
    firePower ← 2
```

```
ELSE
```

```
    firePower ← 1
```

```
ENDIF
```

```
IF GunHeat = 0
```

```
    DAN sudut meriam sudah cukup akurat
```

```
THEN
```

```
    Tembak dengan firePower
```

```
ENDIF
```

Penjelasan

Ketika radar mendeteksi musuh, Dragon langsung mengunci target menggunakan radar dan mengarahkan meriam ke posisi musuh. Kemudian bot memilih kekuatan tembakan berdasarkan jarak target:

- Dekat → firepower tinggi
- Sedang → firepower sedang
- Jauh → firepower rendah

Strategi greedy yang digunakan adalah memilih aksi serangan yang memberikan peluang kill terbesar saat itu berdasarkan posisi target yang sedang terlihat.

Pseudocode Saat Menabrak Bot (OnHitBot)

```
PROSEDUR OnHitBot  
  
IF GunHeat = 0 THEN  
    Tembak dengan firePower maksimum  
ENDIF
```

Penjelasan

Ketika bertabrakan dengan musuh, jarak target sangat dekat sehingga peluang mengenai target hampir pasti. Oleh karena itu Dragon langsung menembakkan peluru berkekuatan maksimum untuk memperoleh damage sebesar mungkin.

Pseudocode Saat Menabrak Dinding (OnHitWall)

```
PROSEDUR OnHitWall  
  
Belok kanan 90 derajat  
Lanjut bergerak maju
```

Penjelasan

Jika bot mengenai dinding arena, Dragon segera mengubah arah geraknya dan kembali melakukan patroli agar tidak terus menabrak dinding.

Penjelasan Struktur Data, Fungsi, dan Prosedur

Struktur Data

Variabel	Tipe	Fungsi
Margin	Konstanta Double	Menentukan jarak aman dari dinding arena
X, Y	Double	Posisi bot saat ini
Direction	Double	Arah hadap bot
DistanceRemaining	Double	Sisa jarak gerakan yang belum ditempuh
GunHeat	Double	Status panas meriam
ArenaWidth, ArenaHeight	Double	Ukuran arena permainan

Fungsi yang Digunakan

DistanceTo(x, y)

Menghitung jarak antara bot dengan musuh.

Digunakan untuk:

- Menentukan kategori jarak target.
- Menentukan firepower yang akan digunakan.

GunBearingTo(x, y)

Menghitung sudut yang harus ditempuh meriam agar mengarah ke target.

Digunakan untuk:

- Mengunci meriam ke musuh.

RadarBearingTo(x, y)

Menghitung sudut radar menuju posisi target.

Digunakan untuk:

- Mempertahankan radar lock pada musuh.

Prosedur yang Digunakan

Run()

Prosedur utama yang menjalankan perilaku patroli Dragon selama pertandingan.

OnScannedBot()

Menangani proses deteksi musuh, penguncian target, pemilihan firepower, dan penembakan.

OnHitBot()

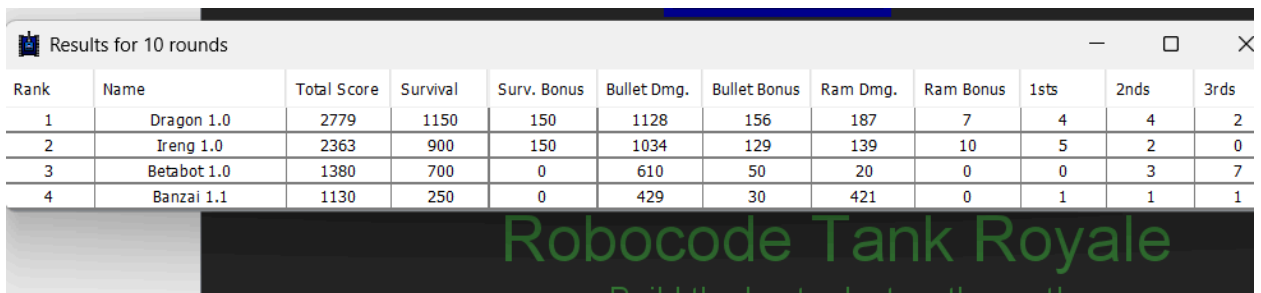
Menangani kondisi tabrakan dengan musuh dan melakukan serangan jarak dekat.

OnHitWall()

Menangani kondisi tabrakan dengan dinding dan mengubah arah pergerakan.

C. Analisis Pengujian

Hasil Pengujian 1



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Dragon 1.0	2779	1150	150	1128	156	187	7	4	4	2
2	Ireng 1.0	2363	900	150	1034	129	139	10	5	2	0
3	Betabot 1.0	1380	700	0	610	50	20	0	0	3	7
4	Banzai 1.1	1130	250	0	429	30	421	0	1	1	1

Robocode Tank Royale
Build the best, destroy the rest!

Hasil Pengujian 2

Results for 8 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Dragon 1.0	1830	750	90	706	67	192	24	2	4	1
2	Ireng 1.0	1730	800	120	674	77	59	0	5	1	0
3	Betabot 1.0	1325	550	0	620	20	112	22	0	2	5
4	Banzai 1.1	1031	300	30	396	20	272	11	1	1	2

Hasil Pengujian 3

Results for 6 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Dragon 1.0	1624	750	90	574	65	113	32	2	2	2
2	Banzai 1.1	1381	400	30	537	71	293	50	2	3	1
3	Ireng 1.0	1141	650	60	396	31	4	0	2	1	3
4	Betabot 1.0	436	0	0	310	0	126	0	0	0	0

Analisis

Bot dragon berhasil memenangkan pertandingan 3/3 yang memberikan persentase kemenangan bot Dragon sebesar 100% dalam percobaan ini. Dan dapat dilihat juga bahwa Bot dragon memiliki sebuah pola dalam setiap pertandingan. Dia hampir selalu memiliki skor bullet dmg terbesar di antara bot bot lain. Dengan berdasarkan hasil pengujian diatas. Hasil algoritma greedy kami sudah berhasil diterapkan

BAB V : Kesimpulan dan Saran

5.1 Kesimpulan

Dengan mengerjakan Tugas Besar Strategi algoritma ini kami telah berhasil membuat bot yang mengaplikasikan algoritma greedy kepada bot tank robocode tank royale. Program final yang mengutamakan keuntungan kill skor

5.2 Saran

Bot yang kami buat dapat dikembangkan ide ide lanjutan, misalnya menerka beberapa kondisi agar dan mencari tempat yang lebih aman tau menjauh dari tempat yang ramai tank agar memaksimalkan survival

Lampiran

Repositori github: https://github.com/MA-D-A/Tubes_MUsangking/

Video Bonus Pengujian: <https://youtu.be/cKjz8XxKf6Y?si=BK-fh1msVoNSalVx>

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.	✓	

Daftar Pustaka

[https://id.wikipedia.org/wiki/Heuristik_\(Ilmu_komputer\).](https://id.wikipedia.org/wiki/Heuristik_(Ilmu_komputer).)

[https://robocode.dev/.](https://robocode.dev/)

[https://id.wikipedia.org/wiki/Algoritma.](https://id.wikipedia.org/wiki/Algoritma)

[https://en.wikipedia.org/wiki/Greedy_algorithm.](https://en.wikipedia.org/wiki/Greedy_algorithm)